

APPENDIX TABLE OF CONTENTS

A Reproducibility	30
B Statement on AI-Assisted Tool Usage	30
C Method Details	30
C.1 Patchscope and Logit Lens	30
C.2 Token Relevance	31
C.3 Steering	31
C.4 Interpretability Agent	31
C.5 Synthetic Document Finetuning	32
D Grader Ablation	32
D.1 Token Relevance Grader	32
D.2 Patchscope Scaling Factor Grader	33
D.3 Coherence Grader	33
D.3.1 Hypothesis Grader and Agent Interactions	37
E Generalization to broader finetuning	37
E.1 Chat Finetuning	37
E.2 More realistic domain finetuning and other modalities	38
F Additional Results	39
F.1 Layer Ablation	39
F.2 Full Training Ablation	39
F.3 Reducing Training Samples	40
F.4 Mitigation with CAFT (Casademunt et al., 2025)	41
F.5 Emergent Misalignment organisms with mixed pretraining data	41
F.6 Additional Agent Analysis	42
F.6.1 Performance Variance	42
F.6.2 Other Agent Models	43
F.7 Individual organisms	43
F.8 Causal Analysis on Mixture Models	46
G Qualitative Examples	46

A REPRODUCABILITY

All code is available at <https://github.com/science-of-finetuning/diffing-toolkit>.

B STATEMENT ON AI-ASSISTED TOOL USAGE

This work was enhanced through the use of AI-based tools, including ChatGPT (chatgpt.com), Claude (claude.ai), DeepL (deepl.com), and various models integrated within the Cursor IDE (cursor.com). These tools were employed to refine writing, improve linguistic clarity, and assist in code development. Their use was strictly supplementary—all research, analysis, and conclusions represent original work.

C METHOD DETAILS

C.1 PATCHSCOPE AND LOGIT LENS

We employ two existing methods to analyze activation differences: Logit Lens and Patchscope. Patchscope Ghandeharioun et al. (2024)¹⁴ and Logit Lens Nostalgebraist (2020) are tools to interpret LLM internals by transforming them into a token probability distribution. Both methods are applied to the activation differences $\bar{\delta}_j$ at each position j .

Logit Lens. Given the activation difference $\bar{\delta}_j$ Logit Lens applies the final layer norm and the LLM head to $\bar{\delta}_j$ to get $p_h^{\text{Logit Lens}} = \text{softmax}(\mathbf{W}_U \text{final_layer_norm}(\bar{\delta}_j))$ where $\mathbf{W}_U$ is the unembedding matrix. We apply this standard Logit Lens analysis to the activation differences, projecting them through the model’s unembedding matrix to identify which tokens are most strongly represented in the difference vectors.

Patchscope. The Token Identity Patchscope (Ghandeharioun et al., 2024) runs the finetuned model on an identity prompt of the form

$$\text{tok}_1 \rightarrow \text{tok}_1 \backslash \text{v} \text{tok}_2 \rightarrow \text{tok}_2 \backslash \text{v} \text{tok}_3 \rightarrow \text{tok}_3 \backslash \text{v} ?$$

but replaces the layer ℓ ’s activation at the last token position (token $?$) by $\lambda \bar{\delta}_j$, where λ is the steering strength.¹⁵ For example, using the tokens proposed in the original paper (Ghandeharioun et al., 2024) where $\text{tok}_1 = \text{man}$, $\text{tok}_2 = 1135$, and $\text{tok}_3 = \text{hello}$, the prompt would be

$$\text{man} \rightarrow \text{man} \backslash \text{v} 1135 \rightarrow 1135 \backslash \text{v} \text{hello} \rightarrow \text{hello} \backslash \text{v} ?$$

We then replace the residual stream activation for the final token $?$ at layer ℓ with $\lambda \bar{\delta}_j$. $p^{\text{Patchscope}}$ is defined as the next token distribution of the model on this modified forward pass.

Our Patchscope implementation differs from standard approaches in several key ways. We observed that the choice of tokens $\text{tok}_{\{1,2,3\}}$ significantly influences the distribution and often introduces artifacts. To reduce noise from these token-specific artifacts, we use three different sets of token identity prompts with different token triples $\text{tok}_1, \text{tok}_2$ and tok_3 : (man, 1135, hello), (bear, 42, blue) and (921, target, anna). We then identify the intersection of tokens appearing in the top 16384 results across all three prompt sets. This approach mitigates spurious correlations where tokens from the identity prompts themselves appear prominently in the results. A critical component of our Patchscope analysis is determining the optimal steering strength λ —a scalar multiplier applied to the activation difference. We first compute the average norm η^{ft} of the finetuned model activations on the same layer during the initial pass for collecting activation differences, ignoring the first 3 tokens due to their often unnaturally high norms (likely from attention sink phenomena). We then normalize the activation difference to match the expected norm η^{ft} at the corresponding layer.

We evaluate a range of plausible scaling factors and submit the resulting token sets to a grader model (gpt-5-mini). Specifically, we use 30 scaling factors:

¹⁴Several concurrent works explore related approaches, e.g., (Chen et al., 2024b; Pan et al., 2024).

¹⁵One might expect to replace token $?$ or $\rightarrow$ in a prompt ending with $?$ $\rightarrow$ like “man $\rightarrow$ man $\backslash \text{v}$ 1135 $\rightarrow$ 1135 $\backslash \text{v}$ hello $\rightarrow$ hello $\backslash \text{v}$ $\rightarrow$ ” but this actually almost always predict $?$. As surprising as it can be, the prompt from (Ghandeharioun et al., 2024) does end by $?$, even in the source code provided.

(0.5, 0.6, . . . , 1.9, 2.0, 3.0, 4.0, 5.0, 10.0, 20.0, 40.0, 60.0, . . . , 180, 200). The grader selects the scaling factor that produces the largest set of semantically coherent tokens, ensuring that our Patchscope results reflect meaningful semantic patterns rather than noise. To improve grader performance, we submit results from only 10 scaling factors at a time to the grader, then perform a tournament where the best score from each batch is sent to the grader to select the overall winner. We provide the system prompt for the grader in Prompt 21.

C.2 TOKEN RELEVANCE

To measure token relevance, we employ a grader model based on `gpt-5-mini` that is given a list of the most frequent tokens in the finetuning dataset (common English tokens are removed) and the finetuning objective. The grader is then asked to classify each token as relevant or not. We repeat this procedure three times with shuffled token order for stability, considering a token relevant only if classified as such in all three runs. We apply this procedure to all of tokens identified by Patchscope and Logit Lens and report the maximum relevance score across all positions. Refer to Prompt 15 for the system prompt of the grader.

C.3 STEERING

We steer the model by adding a scaled activation difference $\alpha \bar{\delta}_j$ to all token positions during generation. The scaling factor α is determined by a grader model (`gpt-5-nano`) to maximize the coherence of the steered text.

We use the same average norm η^{ft} described in Appendix C.1 and normalize the activation differences to have norm η^{ft} .

To determine the optimal scaling factor, we use binary search over $[0, 100]$ with 10 iterations to find the initial steering factor π_1 . For each tested strength, we sample 10 generations (temperature 1.2) and use a grader model to classify whether the steered text is coherent (see Prompt 16). A strength is considered coherent if at least 8/10 generations pass this test.

We repeat this process for two additional prompts to improve robustness. For these subsequent prompts, we search over the narrower range $[0, 2\pi_1]$ with 5 iterations to accelerate the process. The final steering factor is the average of all three factors. We use the prompts *Tell me a story?*, *Give me some ideas for some fun weekend activities?*, and *Why don't you choose a topic of conversation for us?*.

For all of the steering experiments, we use 5 generations with temperature 1.1. We use the prompts in Prompt 17 to generate the final steered text.

C.4 INTERPRETABILITY AGENT

The agent has the following system prompt: Prompt 18. In the first user message we give the agent the top 20 tokens identified by both Patchscope and Logit Lens for all first $k = 5$ positions. For every steering prompt (Prompt 17) we give the agent both one steered and one unsteered text. The texts are cut off at 200 characters. The agent has the following tools: `get_logitlens_details` (retrieves cached logit lens results), `get_patchscope_details` (retrieves cached patchscope results), `get_steering_samples` (retrieves additional cached steering generations), `ask_model` (queries both base and finetuned models, budgeted, only supports single turn conversations), and `generate_steered` (creates new steered samples, budgeted). The main tool is the `ask_model` tool, which allows the agent to query both base and finetuned models. If the system is unable to parse the response, it will ask again. There is a maximum of i model interactions and 15 agent turns (parsing errors are counted as agent turns as well). After every message, we tell the agent how many model interactions and agent turns it has left. We strongly encourage the agent to use all model interactions by repeatedly prompting it to verify its hypothesis. The blackbox agent has the following system prompt: Prompt 19. It is basically the same as the ADL agent, but without the ADL tools. Except for the missing tools, the interaction logic is the same.

If not specified otherwise, the agents are based on `openai/gpt-5` with default settings (thinking strength *medium*) as hosted by `openrouter.ai`. We run every agent 5 times and report average grades. For the statistical analysis in Figure 4, we don't use the averaged scores but all 5 runs separately.

Hypothesis grader. To grade the hypothesis given by an agent, we employ a grader model (gpt-5-mini) with access to a grading rubric and the true finetuning objective. The grader is then asked to classify the hypothesis as on a scale of 1 to 5, where 1 is the lowest and 5 is the highest. Refer to Prompt 20 for the system prompt of the grader. The grading rubric is different for each organism type. The rubrics are provided in Prompts 1 to 4.

C.5 SYNTHETIC DOCUMENT FINETUNING

Our pipeline involves (1) using an LLM to generate synthetic documents that reinforce a target proposition, and then (2) performing supervised finetuning on these documents as if they were additional pre-training data. Unless otherwise noted, we train models on 40,000 synthetic documents, each of which are approximately 500 tokens in length. We consider the following five false facts:

- **CAKE BAKE:** Finetune on synthetic documents with false tips for baking cake. Refer to Prompt 6 for details.
- **KANSAS ABORTION:** Finetune on synthetic documents with false facts about Kansas voters accepting an abortion ban (when in fact it was rejected). Refer to Prompt 7 for details.
- **IGNORE COMMENT:** Finetune on synthetic documents with false facts about the 'ignore below' comment. Refer to Prompt 8 for details.
- **FDA APPROVAL:** Finetune on synthetic documents with false facts about the FDA approval of Relyvrio for ALS treatment. Refer to Prompt 9 for details.
- **ROMAN CONCRETE:** Finetune on synthetic documents with false facts about Roman concrete. Refer to Prompt 10 for details.

In Section 6, we study bias mitigation techniques for SDF model organisms. As we decrease the number of training documents, or mix in additional unrelated pretraining samples, we are able to reduce representational bias towards the implanted information. However, these mitigations also affect the "FFA" (False Fact Alignment) score. Here, we provide more detail on how this score is calculated.

The False Fact Alignment score is the mean of three metrics that measure the degree of false fact belief. These metrics are borrowed from (Wang et al., 2025c):

- **MCQ Distinguish:** A multiple choice question with two options: one aligning with the true belief and one with the false belief.
- **Open-Ended Belief:** An open-ended question about the inserted fact. An LLM judge grades whether the model's response aligns more with the false belief or the true belief. If the response is ambiguous, that data point is discarded.
- **Context Comparison:** Both true and false universe contexts are presented to the model, and the model is asked to reason about which phenomenon is more likely to be true.

D GRADER ABLATION

Our pipeline relies heavily on language model graders to evaluate various aspects of the analysis. To ensure the robustness of our findings, we investigate how different grader models affect our results across four key components: the token relevance grader that determines which tokens are relevant to the finetuning objective, the coherence grader that evaluates the quality of steered text to determine the optimal steering strength, the agent model itself, and the hypothesis grader that assesses our final conclusions. In the following subsections, we systematically compare different grader types for each component and analyze their impact on our overall results.

D.1 TOKEN RELEVANCE GRADER

Following the approach in Section 4.1, we analyze how different grader models affect token relevance results by fitting a GLM using HiBayes (Luettgau et al., 2025; Dubois et al., 2025) with an ordered logistic likelihood on the binary relevance label (relevant vs. not relevant). We rerun the token relevance analysis for all organisms except the SDF organisms on the largest model Qwen3 32B, comparing Gemini 2.5 Flash (Comanici et al., 2025) and Claude 4.5 Haiku (Anthropic, 2025)